sgp4 2.27

The C++ SGP4 routine that, given an Earth satellite TLE, computes its position.

```
pip install sgp4
```

This package compiles the official C++ code from Revisiting Spacetrack Report #3 (AIAA 2006-6753) — specifically, the 2023 May 09 release from David Vallado's Fundamentals of Astrodynamics and Applications webpage — and uses it to compute the positions of satellites in Earth orbit. Satellite orbital elements can be loaded from either a legacy TLE file or from a modern OMM element set, both of which you can fetch from a site like CelesTrak.

What we today call 'SGP4' in fact merges together what in the 1970s were originally two separate satellite propagation routines, but which are now selected automatically:

- SGP4 — the 'Simplified General Perturbations' model is used for satellites close enough to Earth that their orbit takes less than 225 minutes (3 hours 45 minutes) to complete.
- SDP4 — the 'Simplified Deep Space Perturbations' model is used for satellites farther from Earth, which take 225 minutes or longer to compete an orbit.

If your machine can't install or compile the C++ code, then this package falls back to using a slower pure-Python implementation of the library. Tests make sure that its positions **agree to within 0.1 mm** with the standard version of the algorithm — an error far less than the 1–3 km/day by which satellites themselves deviate from the ideal orbits described in TLE files.

An accelerated routine is available that, given a series of times and of satellites, computes a whole array of output positions using a fast C++ loop. See the "Array Acceleration" section below.

Note that the SGP4 propagator returns raw *x,y,z* Cartesian coordinates in a "True Equator Mean Equinox" (TEME) reference frame that's centered on the Earth but does not rotate with it — an "Earth centered inertial" (ECI) reference frame. The SGP4 propagator itself does not implement the math to convert these positions into more official ECI frames like J2000 or the ICRS, nor into any Earth-centered Earth-fixed (ECEF) frames like the ITRS, nor into latitudes and longitudes through an Earth ellipsoid like WGS84. For conversions into these other coordinate frames, look for a comprehensive astronomy library, like the Skyfield library that is built atop this one (see the section on Earth satellites in its documentation).

Usage

You will probably first want to to check whether your machine has successfully installed the fast SGP4 C++ code, or is using the slow Python version (in which case this value will be false):

```
>>> from sgp4.api import accelerated
>>> print(accelerated)
True
```

This library uses the same function names as the official C++ code, to help users who are already familiar with SGP4 in other languages. Here is how to compute the x,y,z position and velocity for the International Space Station at 20:42:00 on 2019 December 9:

```
>>> from sgp4.api import Satrec
>>>
>>> s = '1 25544U 98067A   19343.69339541  .00001764  00000-
>>> t = '2 25544  51.6439 211.2001 0007417  17.6667  85.6398
>>> satellite = Satrec.twoline2rv(s, t)
>>>
>>> jd, fr = 2458826.5, 0.8625
>>> e, r, v = satellite.sgp4(jd, fr)
>>> e
0
>>> print(r)  # True Equator Mean Equinox position (km)
(-6088.92..., -936.13..., -2866.44...)
>>> print(v)  # True Equator Mean Equinox velocity (km/s)
(-1.525..., -5.538..., 5.068...)
```

As input, you can provide either:

- A simple floating-point Julian Date for `jd` and the value 0.0 for `fr`, if you are happy with the precision of a 64-bit floating point number. Note that modern Julian Dates are greater than 2,450,000 which means that nearly half of the precision of a 64-bit float will be consumed by the whole part that specifies the day. The remaining digits will provide a precision for the fraction of around 20.1 µs. This should be no problem for the accuracy of your result — satellite positions are usually off by a few kilometers anyway, far less than a satellite moves in 20.1 µs — but if you run a solver that dives down into the microseconds while searching for a rising or setting time, the solver might be bothered by the 20.1 µs plateau between each jump in the satellite's position.
- Or, you can provide a coarse date `jd` plus a very precise fraction `fr` that supplies the rest of the value. The Julian Date for which the satellite position is computed is the sum of the two values. One common practice is to provide the whole number as `jd` and the fraction as `fr`; another is to have `jd` carry the fraction 0.5 since UTC midnight occurs halfway through each Julian Date. Either way, splitting the value allows a solver to run all the way down into the nanoseconds and still see SGP4 respond smoothly to tiny date adjustments with tiny changes in the resulting satellite position.

Here is how to intrepret the results:

- `e` will be a non-zero error code if the satellite position could not be computed for the given date. You can `from sgp4.api import SGP4_ERRORS` to access a dictionary mapping error codes to error messages explaining what each code means.
- `r` measures the satellite position in **kilometers** from the center of the earth in the idiosyncratic True Equator Mean Equinox coordinate frame used by SGP4.
- `v` velocity is the rate at which the position is changing, expressed in **kilometers per second**.

If your application does not natively handle Julian dates, you can compute `jd` and `fr` from calendar dates using `jday()`.

```
>>> from sgp4.api import jday
>>> jd, fr = jday(2019, 12, 9, 20, 42, 0)
>>> jd
2458826.5
>>> fr
0.8625
```

Double-checking your TLE lines

Because TLE is an old punch-card fixed-width format, it's very sensitive to whether exactly the right number of spaces are positioned in exactly the right columns. If you suspect that your satellite elements aren't getting loaded correctly, try calling the slow pure-Python version of `twoline2rv()`, which performs extra checks that the fast C++ doesn't:

```
>>> from sgp4.earth_gravity import wgs72
>>> from sgp4.io import twoline2rv
>>> assert twoline2rv(s, t, wgs72)
```

Any TLE formatting errors will be raised as a `ValueError`.

Using OMM elements instead of TLE

The industry is migrating away from the original TLE format, because it will soon run out of satellite numbers.

- Some TLE files now use a new "Alpha-5" convention that expands the range of satellite numbers by using an initial letter; for example, "E8493" means satellite 148493. This library supports the Alpha-5 convention and should return the correct integer in Python.
- Some authorities are now distributing satellite elements in an "OMM" Orbit Mean Elements Message format that replaces the TLE format. You can learn about OMM in Dr. T.S. Kelso's "A New Way to Obtain GP Data" at the CelesTrak site.

You can already try out experimental support for OMM:

```
>>> from sgp4 import omm
```

Reading OMM data takes two steps, because OMM supports several different text formats. First, parse the input text to recover the field names and values that it stores; second, build a Python satellite object from those field values. The fastest and most efficient format is usually CSV, since the column names are only given once, on the first line:

```
>>> with open('sample_omm.csv') as f:
...     for fields in omm.parse_csv(f):
...         sat = Satrec()
...         omm.initialize(sat, fields)
```

The JSON format is more verbose because it repeats the field names over again in every single record:

```
>>> import json
>>> with open('sample_omm.json') as f:
...     record_list = json.load(f)
>>> for fields in record_list:
...     sat = Satrec()
...     omm.initialize(sat, fields)
```

The most verbose format is XML:

```
>>> with open('sample_omm.xml') as f:
...     for fields in omm.parse_xml(f):
...         sat = Satrec()
...         omm.initialize(sat, fields)
```

Either way, the satellite object should wind up properly initialized and ready to start producing positions.

The `omm.initialize()` routine uses the standard WGS72 gravity model by default. To make your own choice of model, supply a third argument that is one of the three constants `WGS72OLD`, `WGS72`, or `WGS84` (which are shown below in the section on 'Gravity').

If you are interested in saving satellite parameters using the new OMM format, then read the section on "Export" below.

Epoch

Over a given satellite's lifetime, dozens or hundreds of different TLE records will be produced as its orbit evolves. Each TLE record specifies the "epoch date" for which it is most accurate. Typically a TLE is only useful for a couple of weeks to either side of its epoch date, beyond which its predictions become unreliable.

Satellite objects natively provide their epoch as a two-digit year and then a fractional number of days into the year:

```
>>> satellite.epochyr
19
>>> satellite.epochdays
343.69339541
```

Because Sputnik was launched in 1957, satellite element sets will never refer to an earlier year, so years 57 through 99 mean 1957–1999 while 0 through 56 mean 2000–2056. The TLE format will presumably be obsolete in 2057 and have to be upgraded to 4-digit years.

To turn the number of days and its fraction into a calendar date and time, use the `days2mdhms()` function.

```
>>> from sgp4.api import days2mdhms
>>> month, day, hour, minute, second = days2mdhms(19, 343.69
>>> month
12
>>> day
9
>>> hour
16
>>> minute
38
>>> second
29.363424
```

The SGP4 library also translates those two numbers into a Julian date and fractional Julian date, since Julian dates are more commonly used in astronomy.

```
>>> satellite.jdsatepoch
2458826.5
>>> satellite.jdsatepochF
0.69339541
```

Finally, a convenience function is available in the library if you need the epoch date and time as Python `datetime`.

```
>>> from sgp4.conveniences import sat_epoch_datetime
>>> sat_epoch_datetime(satellite)
datetime.datetime(2019, 12, 9, 16, 38, 29, 363423, tzinfo=UT
```

Array Acceleration

To avoid the expense of running a Python loop when you have many dates and times for which you want a position, you can pass your Julian dates as arrays. The array routine is only faster if your machine has successfully installed or compiled the SGP4 C++ code, so you might want to check first:

```
>>> from sgp4.api import accelerated
>>> print(accelerated)
True
```

To call the array routine, make NumPy arrays for `jd` and `fr` that are the same length:

```
>>> import numpy as np
>>> np.set_printoptions(precision=2)
```

```
>>> jd = np.array((2458826, 2458826, 2458826, 2458826))
>>> fr = np.array((0.0001, 0.0002, 0.0003, 0.0004))
```

```
>>> e, r, v = satellite.sgp4_array(jd, fr)
```

```
>>> print(e)
[0 0 0 0]
>>> print(r)
[[-3431.31  2620.15 -5252.97]
 [-3478.86  2575.14 -5243.87]
 [-3526.09  2529.89 -5234.28]
 [-3572.98  2484.41 -5224.19]]
>>> print(v)
[[-5.52 -5.19  1.02]
 [-5.49 -5.22  1.08]
 [-5.45 -5.25  1.14]
 [-5.41 -5.28  1.2 ]]
```

To avoid the expense of Python loops when you have many satellites and dates, build a `SatrecArray` from several individual satellites. Its `sgp4()` method will expect both `jd` and `fr` to be NumPy arrays, so if you only have one date, be sure to provide NumPy arrays of length one. Here is a sample computation for 2 satellites and 4 dates:

```
>>> u = '1 20580U 90037B   19342.88042116  .00000361  00000-
>>> w = '2 20580  28.4682 146.6676 0002639 185.9222 322.7238
>>> satellite2 = Satrec.twoline2rv(u, w)
```

```
>>> from sgp4.api import SatrecArray
>>> a = SatrecArray([satellite, satellite2])
>>> e, r, v = a.sgp4(jd, fr)
```

```
>>> np.set_printoptions(precision=2)
>>> print(e)
[[0 0 0 0]
 [0 0 0 0]]
>>> print(r)
[[[-3431.31  2620.15 -5252.97]
  [-3478.86  2575.14 -5243.87]
  [-3526.09  2529.89 -5234.28]
  [-3572.98  2484.41 -5224.19]]
<BLANKLINE>
 [[ 5781.85  2564.   -2798.22]
  [ 5749.36  2618.59 -2814.63]
  [ 5716.35  2672.94 -2830.78]
  [ 5682.83  2727.05 -2846.68]]]
>>> print(v)
[[[-5.52 -5.19  1.02]
  [-5.49 -5.22  1.08]
  [-5.45 -5.25  1.14]
  [-5.41 -5.28  1.2 ]]
<BLANKLINE>
 [[-3.73  6.33 -1.91]
  [-3.79  6.3  -1.88]
  [-3.85  6.28 -1.85]
  [-3.91  6.25 -1.83]]]
```

Export

If you have a `Satrec` you want to share with friends or persist to a file, there's an export routine that will turn it back into a TLE:

```
>>> from sgp4 import exporter
>>> line1, line2 = exporter.export_tle(satellite)
>>> line1
'1 25544U 98067A   19343.69339541  .00001764  00000-0  38792
>>> line2
'2 25544  51.6439 211.2001 0007417  17.6667  85.6398 15.5010
```

Happily, these are exactly the two TLE lines that we used to create this satellite object:

```
>>> (s == line1) and (t == line2)
True
```

Another export routine is available that produces the fields defined by the new OMM format (see the "OMM" section above):

```
>>> from pprint import pprint
>>> fields = exporter.export_omm(satellite, 'ISS (ZARYA)')
>>> pprint(fields)
{'ARG_OF_PERICENTER': 17.6667,
 'BSTAR': 3.8792e-05,
 'CENTER_NAME': 'EARTH',
 'CLASSIFICATION_TYPE': 'U',
 'ECCENTRICITY': 0.0007417,
 'ELEMENT_SET_NO': 999,
 'EPHEMERIS_TYPE': 0,
 'EPOCH': '2019-12-09T16:38:29.363423',
 'INCLINATION': 51.6439,
 'MEAN_ANOMALY': 85.6398,
 'MEAN_ELEMENT_THEORY': 'SGP4',
 'MEAN_MOTION': 15.501034720000002,
 'MEAN_MOTION_DDOT': 0.0,
 'MEAN_MOTION_DOT': 1.764e-05,
 'NORAD_CAT_ID': 25544,
 'OBJECT_ID': '1998-067A',
 'OBJECT_NAME': 'ISS (ZARYA)',
 'RA_OF_ASC_NODE': 211.2001,
 'REF_FRAME': 'TEME',
 'REV_AT_EPOCH': 20248,
 'TIME_SYSTEM': 'UTC'}
```

Gravity

The SGP4 algorithm operates atop a set of constants specifying how strong the Earth's gravity is. The most recent official paper on SGP4 (see below) specifies that "We use WGS-72 as the default value", so this Python module uses the same default. But in case you want to use either the old legacy version of the WGS-72 constants, or else the non-standard but more modern WGS-84 constants, the `twoline2rv()` constructor takes an optional argument:

```
>>> from sgp4.api import WGS72OLD, WGS72, WGS84
>>> satellite3 = Satrec.twoline2rv(s, t, WGS84)
```

You will in general get less accurate results if you choose WGS-84. Even though it reflects more recent and accurate measures of the Earth, satellite TLEs across the industry are most likely generated with WGS-72 as their basis. The positions you generate will better agree with the real positions of each satellite if you use the same underlying gravity constants as were used to generate the TLE.

Providing your own elements

If instead of parsing a TLE you want to specify orbital elements directly, you can pass them as floating point numbers to a satellite object's `sgp4init()` method. For example, here's how to build the same International Space Station orbit that we loaded from a TLE in the first code example above:

```
>>> satellite2 = Satrec()
>>> satellite2.sgp4init(
...     WGS72,                 # gravity model
...     'i',                   # 'a' = old AFSPC mode, 'i' =
...     25544,                 # satnum: Satellite number
...     25545.69339541,        # epoch: days since 1949 Decem
...     3.8792e-05,            # bstar: drag coefficient (1/e
...     0.0,                   # ndot: ballistic coefficient
...     0.0,                   # nddot: mean motion 2nd deriv
...     0.0007417,             # ecco: eccentricity
...     0.3083420829620822,    # argpo: argument of perigee (
...     0.9013560935706996,    # inclo: inclination (radians
...     1.4946964807494398,    # mo: mean anomaly (radians 0.
...     0.06763602333248933,   # no_kozai: mean motion (radia
...     3.686137125541276,     # nodeo: R.A. of ascending noc
... )
```

You might notice that these numbers don't look the same as the numbers in the TLE above. For example, the inclination was 51.6439 in the TLE, but is 0.901356 here. That's because `sgp4init()` uses different units than the TLE format: angles are in radians rather than degrees. But this is the same orbit and will produce the same positions.

If you want to double-check that your elements are valid, you can run the function `check_satrec()`, which will raise a `ValueError` with an informative error message if any of the angles are out of bounds. If the satellite is fine, it simply returns, without raising an exception:

```
>>> from sgp4.conveniences import check_satrec
>>> check_satrec(satellite2)
```

Note that `ndot` and `nddot` are ignored by the SGP4 propagator, so you can leave them `0.0` without any effect on the resulting satellite positions. But they do at least get saved to the satellite object, and written out if you write the parameters to a TLE or OMM file (see the "Export" section, above).

To compute the "epoch" argument, take the epoch's Julian date and subtract 2433281.5 days.

While the underlying `sgp4init()` routine leaves the attributes `epochyr`, `epochdays`, `jdsatepoch`, and `jdsatepochF` unset, this library goes ahead and sets them anyway for you, using the epoch you provided.

See the next section for the complete list of attributes that are available from the satellite record once it has been initialized.

Attributes

There are several dozen `Satrec` attributes that expose data from the underlying C++ SGP4 record. They fall into the following categories.

Identification

These are copied directly from the TLE record but aren't used by the propagation math.
`satnum_str` — Satellite number, as a 5-character string.
`satnum` — Satellite number, converted to an integer.
`classification` — `'U'`, `'C'`, or `'S'` indicating the element set is Unclassified, Classified, or Secret.
`ephtype` — Integer "ephemeris type", used internally by space agencies to mark element sets that are not ready for publication; this field should always be `0` in published TLEs.
`elnum` — Element set number.
`revnum` — Satellite's revolution number at the moment of the epoch, presumably counting from 1 following launch.

Orbital Elements

These are the orbital parameters, copied verbatim from the text of the TLE record. They describe the orbit at the moment of the TLE's epoch and so remain constant even as the satellite record is used over and over again to propagate positions for different times.
`epochyr` — Epoch date: the last two digits of the year.
`epochdays` — Epoch date: the number of days into the year, including a decimal fraction for the UTC time of day.
`ndot` — First time derivative of the mean motion (loaded from the TLE, but otherwise ignored).
`nddot` — Second time derivative of the mean motion (loaded from the TLE, but otherwise ignored).
`bstar` — Ballistic drag coefficient B* (1/earth radii).
`inclo` — Inclination (radians $0 \le i < pi$).
`nodeo` — Right ascension of ascending node (radians $0 \le \Omega < 2pi$).
`ecco` — Eccentricity.
`argpo` — Argument of perigee (radians $0 \le \omega < 2pi$).
`mo` — Mean anomaly (radians $0 \le M < 2pi$).
`no_kozai` — Mean motion (radians/minute).
`no` — Alias for `no_kozai`, for compatibility with old code.

You can also access the epoch as a Julian date:
`jdsatepoch` — Whole part of the epoch's Julian date.
`jdsatepochF` — Fractional part of the epoch's Julian date.

Computed Orbit Properties

These are computed when the satellite is first loaded, as a convenience for callers who might be interested in them. They aren't used by the SGP4 propagator itself.
`a` — Semi-major axis (earth radii).
`altp` — Altitude of the satellite at perigee (earth radii, assuming a spherical Earth).
`alta` — Altitude of the satellite at apogee (earth radii, assuming a spherical Earth).
`argpdot` — Rate at which the argument of perigee is changing (radians/minute).
`gsto` — Greenwich Sidereal Time at the satellite's epoch (radians).
`mdot` — Rate at which the mean anomaly is changing (radians/minute)
`nodedot` — Rate at which the right ascension of the ascending node is changing (radians/minute).

Propagator Mode

`operationmode` — A single character that directs SGP4 to either operate in its modern `'i'` improved mode or in its legacy `'a'` AFSPC mode.
`method` — A single character, chosen automatically when the orbital elements were loaded, that indicates whether SGP4 has chosen to use its built-in `'n'` Near Earth or `'d'` Deep Space mode for this satellite.

Result of Most Recent Propagation

`t` — The time you gave when you most recently asked SGP4 to compute this satellite's position, measured in minutes before (negative) or after (positive) the satellite's epoch.
`error` — Error code produced by the most recent SGP4 propagation you performed with this element set.

The possible `error` codes are:

0. No error.
1. Mean eccentricity is outside the range $0 \le e < 1$.
2. Mean motion has fallen below zero.
3. Perturbed eccentricity is outside the range $0 \le e \le 1$.
4. Length of the orbit's semi-latus rectum has fallen below zero.
5. (No longer used.)
6. Orbit has decayed: the computed position is underground. (The position is still returned, in case the vector is helpful to software that might be searching for the moment of re-entry.)

Mean Elements From Most Recent Propagation

Each time the SGP4 routine generates a satellite position, it also saves a set of "singly averaged mean elements" that describe shape of the satellite's orbit at that moment. They are averaged with respect to the mean anomaly and include the effects of secular gravity, atmospheric drag, and — in Deep Space mode — of those pertubations from the Sun and Moon that SGP4 averages over an entire revolution of each of those bodies. They omit both the shorter-term and longer-term periodic pertubations from the Sun and Moon that SGP4 applies right before computing each position.
`am` — Average semi-major axis (earth radii).
`em` — Average eccentricity.
`im` — Average inclination (radians).
`Om` — Average right ascension of ascending node (radians).
`om` — Average argument of perigee (radians).
`mm` — Average mean anomaly (radians).
`nm` — Average mean motion (radians/minute).

Gravity Model Parameters

When the satellite record is initialized, your choice of gravity model results in a slate of eight constants being copied in:
`tumin` — Minutes in one "time unit".
`xke` — The reciprocal of `tumin`.
`mu` — Earth's gravitational parameter (km^3/s^2).
`radiusearthkm` — Radius of the earth (km).
`j2`, `j3`, `j4` — Un-normalized zonal harmonic values J_2, J_3, and J_4.
`j3oj2` — The ratio J_3/J_2.

Printing satellite attributes

If you want to print out a satellite, this library provides a convenient "attribute dump" routine that takes a satellite and generates lines that list its attributes:

```
from sys import stdout
from sgp4.conveniences import dump_satrec

stdout.writelines(dump_satrec(satellite))
```

If you want to compare two satellites, then simply pass a second argument; the second satellite's attributes will be printed in a second column next to those of the first.

```
stdout.writelines(dump_satrec(satellite, satellite2))
```

Validation against the official algorithm

This implementation passes all of the automated tests in the August 2010 release of the reference implementation of SGP4 by Vallado et al., who originally published their revision of SGP4 in 2006:

> *Vallado, David A., Paul Crawford, Richard Hujsak, and T.S. Kelso, "Revisiting Spacetrack Report #3," presented at the AIAA/AAS Astrodynamics Specialist Conference, Keystone, CO, 2006 August 21–24.*

If you would like to review the paper, it is available online. You can always download the latest version of their code for comparison against this Python module (or other implementations) at AIAA-2006-6753.zip.

For developers

Developers can check out this full project from GitHub:

https://github.com/brandon-rhodes/python-sgp4

To run its unit tests, install Python 2, Python 3, and the `tox` testing tool. The tests runing in Python 2 will exercise the fallback pure-Python version of the routines, while Python 3 exercises the fast new C++ accelerated code:

```
cd python-sgp4
tox
```

Legacy API

Before this library pivoted to wrapping Vallado's official C++ code and was operating in pure Python only, it had a slightly quirkier API, which is still supported for compatibility with older clients. You can learn about it by reading the documentation from version 1.4 or earlier:

https://pypi.org/project/sgp4/1.4/

Changelog

2026-07-03 — 2.27

- Moved the project documentation out of the `__init__.py` docstring and into the source code README file, which should hopefully make the documentation reappear on the project's PyPI page.

2026-06-26 — 2.26

- The C++ module now limits itself to calling the Python Limited API and can therefore link against the Python Stable ABI, producing a binary wheel that's compatible with multiple versions of Python. Hopefully this means that new Python releases no longer require sgp4 to be immediately re-released to support them.

2025-08-04 — 2.25

- Added a `gravconst` parameter to the `omm.initialize()` routine for folks using gravity models other than `WGS72`, or who at least want to make that choice explicit in their code.
- Added a `revnum` attribute to satellites built with `sgp4init()` (it defaults to the dummy value of zero), so they can be passed to `export_tle()` without raising an exception.

2024-02-15 — 2.24

- The documentation now specifies the acceptable range for orbital element angles like inclination and mean anomaly, and a new function `check_satrec(sat)` will tell the caller if any of the angles are out of bounds.
- The documentation now gives an example of loading elements from JSON.
- Tweaked the fallback Python code to accept TLE lines without a final checksum character in the 69th column, to match the C++ code.

2023-10-01 — 2.23

- Tweaked tests to resolve breakage introduced by Python 3.12.

2023-04-27 — 2.22

- Added a `satnum_str` attribute, exposing the fact that the C++ now stores the satellite number as a string; and check that `satnum` is never greater than 339999.
- Fixed the units of the `nddot` attribute when the value is loaded from an OMM record. (Since the TLE computation itself ignores this attribute, this did not affect any satellite positions.)
- Enhanced the fallback Python version of `twoline2rv()` to verify that TLE lines are ASCII, and added documentation using it to double-check TLEs that might suffer from non-ASCII characters.
- If the user doesn't set a satellite's `classification`, it now defaults to `'U'` for 'unclassified'.

2022-04-06 — 2.21

- Added `dump_satrec()` to the `sgp4.conveniences` module.
- Fixed the `Satrec` attribute `.error`, which was previously building a nonsense integer from the wrong data in memory.
- Removed `.whichconst` from Python `Satrec`, to help users avoid writing code that will break when the C++ extension is available.

2021-07-01 — 2.20

- Taught `sgp4init()` to round both `epochdays` and `jdsatepochF` to the same 8 decimal places used for the date fraction in a TLE, if the user-supplied `epoch` itself has 8 or fewer digits behind the decimal point. This should make it easier to build satellites that round-trip to TLE format with perfect accuracy.
- Fixed how `export_tle()` formats the BSTAR field when its value, if written in scientific notation, has a positive exponent.
- Fixed the `epochyr` assigned by `sgp4init()` so years before 2000 have two digits instead of three (for example, so that 1980 produces an `epochyr` of 80 instead of 980).

2021-04-22 — 2.19

- Extended the documentation on the Python Package Index and in the module docstring so it lists every `Satrec` attribute that this library exposes; even the more obscure ones might be useful to folks working to analyze satellite orbits.

2021-03-08 — 2.18

- If a TLE satellite number lacks the required 5 digits, `twoline2rv()` now gives the underlying C++ library a little help so it can still parse the classification and international designator correctly.
- The `Satrec` attributes `jdsatepoch`, `jdsatepochF`, `epochyr`, and `epochdays` are now writeable, so users can adjust their values manually — which should make up for the fact that the `sgp4init()` method can't set them with full floating point precision.

2021-02-17 — 2.17 — Fixed where in the output array the `sgp4_array()` method writes NaN values when an SGP4 propagation fails.
2021-02-12 — 2.16 — Fixed `days2mdhms()` rounding to always match TLE epoch.
2021-01-08 — 2.15 — Fixed parsing of the `satnum` TLE field in the Python fallback code, when the field has a leading space; added OMM export routine.
2020-12-16 — 2.14 — New data formats: added OMM message support for both XML and CSV, and added support for the new Alpha-5 extension to TLE files.
2020-10-14 — 2.13 — Enhanced `sgp4init()` with custom code that also sets the `epochdays` and `epochyr` satellite attributes.
2020-05-28 — 2.12 — Moved the decision of whether to set the locale during `twoline2rv()` from import time to runtime, for users who change locales after their application is up and running.
2020-05-24 — 2.11 — Fixed a regression in how dates are split into hours, minutes, and seconds that would sometimes produce a time whose second=60, crashing the pure-Python version of the library.
2020-05-22 — 2.10 — Switch the locale temporarily to `C` during the C++ accelerated `twoline2rv()`, since it does not protect its `sscanf()` calls from locales that, like German, expect comma decimal points instead of the period decimal points always used in a TLE.
2020-05-21 — 2.9 — Added `sat_epoch_datetime()`, expanded documentation around converting a satellite epoch to a date and time, and started rounding the epoch to exactly the digits provided in the TLE; and removed the `Satrec.epoch` attribute from Python fallback code to better match the C++ version.
2020-05-07 — 2.8 — New function `jday_datetime()` is now available in the `sgp4.conveniences` module, thanks to Egemen Imre.
2020-04-24 — 2.7 — New method `sgp4init()` (thank you, Chris Lewicki!) is available.
2020-04-20 — 2.6 — New routine `export_tle()` (thank you, Egemen Imre!) is available. Improved how the accelerated C++ backend parses the `intldesg` string and the `revnum` integer.
2020-03-22 — 2.5 — Gave the new accelerated `twoline2rv()` an optional argument that lets the user choose a non-standard set of gravity constants.
2020-02-25 — 2.4 — Improved the `jday()` docstring; made the old legacy Python resilient if the day of the month is out-of-range (past the end of the month) in a TLE; and Mark Rutten fixed the C++ so it compiles on Windows!
2020-02-04 — 2.3 — Removed experimental code that caused performance problems for users with Numba installed.
2020-02-02 — 2.2 — A second release on Palindrome Day: fix the Satrec `.epochyr` attribute so it behaves the same way in Python as it does in the official C library, where it is only the last 2 digits of the year; and make `.no` available in the Python fallback case as well.
2020-02-02 — 2.1 — Add vectorized array method to Satrec object; add `.no` attribute to new Satrec object to support old code that has not migrated to the new name `.no_kozai`; gave Python wrapper classes `__slots__` to avoid the expense of a per-object attribute dictionary.
2020-01-30 — 2.0 — Rewrite API to use genuine Vallado C++ code on those systems where it can be compiled; add accelerated vectorized array interface; make `gstime()` a public function; clarify format error message.
2015-01-15 — 1.4 — Display detailed help when TLE input does not match format.
2014-06-26 — 1.3 — Return `(NaN,NaN,NaN)` vectors on error and set `.error_message`
2013-11-29 — 1.2 — Made `epochyr` 4 digits; add `datetime` for `.epoch`
2012-11-22 — 1.1 — Python 3 compatibility; more documentation
2012-08-27 — 1.0 — Initial release

Project links

Homepage

Key dates

PyPI data

Released: Jul 3, 2026

LATEST RELEASE

2 maintainers

PyPI data

brandonrhodes

sgp4-bot

Credits

Author: Brandon Rhodes

License expression

MIT
View SPDX License List

Requires

Python !=3.0.*, !=3.1.*, !=3.2.*, !=3.3.*, !=3.4.*, !=3.5.*, !=3.6.*, !=3.7.*, !=3.8.*, !=3.9.*, >=2.7

Classifiers

Development Status
- 5 - Production/Stable

Intended Audience
- Science/Research

Programming Language
- Python :: 2
- Python :: 2.7
- Python :: 3
- Python :: 3.10
- Python :: 3.11
- Python :: 3.12
- Python :: 3.13
- Python :: 3.14

Topic
- Scientific/Engineering :: Astronomy

SerpApi
SerpApi is a Maintaining sponsor of the Python Software Foundation.
PSF Sponsor · Served ethically

Report project as malware

PyPI
Developed and maintained by the Python Software Foundation and Python community, for the Python community.
Status: All Systems Operational
Donate today!

Help
Installing packages
Uploading packages
User guide
Project name retention
FAQs

About PyPI
PyPI Blog
Infrastructure dashboard
Statistics
Logos & trademarks
Our sponsors

Contributing to PyPI
Bugs and feedback
Contribute on GitHub
Translate PyPI
Sponsor PyPI
Development credits

Using PyPI
Terms of Service
Report security issue
Code of conduct
Privacy Notice
Acceptable Use Policy

Switch to desktop version

Anthropic, PBC Visionary sponsor
Bloomberg Visionary sponsor
Hudson River Trading Visionary sponsor
Meta Visionary sponsor
NVIDIA Visionary sponsor
Microsoft Sustainability sponsor
Depot Continuous Integration
AWS Cloud computing and Security Sponsor
Datadog Monitoring
Fastly CDN
Google Download Analytics
Sentry Error logging
StatusPage Status page

"PyPI", "Python Package Index", and the blocks logos are registered trademarks of the Python Software Foundation.
© 2026 Python Software Foundation   Site map   Deployed from 9448ba6